

```

import os
import time
import logging
import requests
import pandas as pd
from datetime import datetime, timezone

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler("bot/bot.log"),
    ],
)
logger = logging.getLogger(__name__)

BYBIT_API_KEY      = os.environ["BYBIT_API_KEY"]
BYBIT_API_SECRET   = os.environ["BYBIT_API_SECRET"]
TELEGRAM_BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
TELEGRAM_CHAT_ID   = os.environ["TELEGRAM_CHAT_ID"]

PAIRS = [
    {"symbol": "BTCUSDT", "yahoo": "BTC-USD", "name": "BTC"},
    {"symbol": "ETHUSDT", "yahoo": "ETH-USD", "name": "ETH"},
    {"symbol": "SOLUSDT", "yahoo": "SOL-USD", "name": "SOL"},
]

EMA_FAST      = 50
EMA_SLOW      = 200
WILLIAMS_PERIOD = 14
MACD_FAST     = 12
MACD_SLOW     = 26
MACD_SIGNAL    = 9
ATR_PERIOD    = 14
ATR_MULT      = 1.5
RISK_PERCENT   = 2.0
CHECK_INTERVAL = 1800
CANDLE_COUNT   = 250
PAPER_TRADE    = True
WR_OVERSOLD    = -80
WR_OVERBOUGHT  = -20

```

```

def make_state(balance=3333.0):
    return {
        "usdt": balance,
        "coin": 0.0,
        "pos": None,
        "n": 0,
        "wins": 0,
        "loss": 0,
        "pnl": 0.0,
        "log": [],
    }

states = {p["symbol"]: make_state() for p in PAIRS}

def tg(msg):
    url = "https://api.telegram.org/bot" + TELEGRAM_BOT_TOKEN + "/sendMessage"
    try:
        r = requests.post(url, json={"chat_id": TELEGRAM_CHAT_ID, "text": msg, "p
        r.raise_for_status()
        return True
    except Exception as e:
        logger.error("TG error: " + str(e))
        return False

def get_klines(sym):
    try:
        url = "https://query1.finance.yahoo.com/v8/finance/chart/" + sym
        r = requests.get(url, params={"interval": "30m", "range": "60d"},
                        headers={"User-Agent": "Mozilla/5.0"}, timeout=15)
        r.raise_for_status()
        d = r.json()
        res = d["chart"]["result"][0]
        ts = res.get("timestamp", [])
        q = res["indicators"]["quote"][0]
        df = pd.DataFrame({
            "ts": ts,
            "high": q.get("high", []),
            "low": q.get("low", []),
            "close": q.get("close", []),
            "volume": q.get("volume", []),
        })
        df = df.dropna(subset=["close", "high", "low"]).reset_index(drop=True)
        for col in ["close", "high", "low", "volume"]:
            df[col] = df[col].astype(float)

```

```

        if len(df) < EMA_SLOW + 10:
            logger.info("Not enough candles: " + str(len(df)))
            return None
        return df.tail(CANDLE_COUNT).reset_index(drop=True)
except Exception as e:
    logger.error("klines error " + sym + ": " + str(e))
    return None

def get_price(sym):
    try:
        url = "https://query1.finance.yahoo.com/v8/finance/chart/" + sym
        r = requests.get(url, params={"interval": "1m", "range": "1d"},
                        headers={"User-Agent": "Mozilla/5.0"}, timeout=15)
        r.raise_for_status()
        closes = r.json()["chart"]["result"][0]["indicators"]["quote"][0]["close"]
        p = next((c for c in reversed(closes) if c is not None), None)
        return float(p) if p else None
    except Exception as e:
        logger.error("price error: " + str(e))
        return None

def indicators(df):
    df = df.copy()
    df["ef"] = df["close"].ewm(span=EMA_FAST, adjust=False).mean()
    df["es"] = df["close"].ewm(span=EMA_SLOW, adjust=False).mean()
    hh = df["high"].rolling(WILLIAMS_PERIOD).max()
    ll = df["low"].rolling(WILLIAMS_PERIOD).min()
    df["wr"] = ((hh - df["close"]) / (hh - ll).replace(0, 1)) * -100
    mf = df["close"].ewm(span=MACD_FAST, adjust=False).mean()
    ms = df["close"].ewm(span=MACD_SLOW, adjust=False).mean()
    mc = (mf - ms).ewm(span=MACD_SIGNAL, adjust=False).mean()
    df["mh"] = (mf - ms) - mc
    hl = df["high"] - df["low"]
    hc = (df["high"] - df["close"].shift()).abs()
    lc = (df["low"] - df["close"].shift()).abs()
    tr = pd.concat([hl, hc, lc], axis=1).max(axis=1)
    df["atr"] = tr.ewm(span=ATR_PERIOD, adjust=False).mean()
    return df

def signal(df):
    c = df.iloc[-1]
    p = df.iloc[-2]
    up = c["ef"] > c["es"]
    buy = up and p["wr"] <= WR_OVERSOLD and c["wr"] > WR_OVERSOLD and p["mh"] <

```

```

sell = c["wr"] >= WR_OVERBOUGHT or (p["mh"] >= 0 and c["mh"] < 0) or c["ef"]
if buy:
    return "BUY"
if sell:
    return "SELL"
return None

def do_buy(pair, price, atr):
    s = states[pair["symbol"]]
    if s["pos"]:
        return None
    amt = s["usdt"] * (RISK_PERCENT / 100)
    qty = round(amt / price, 6)
    cost = qty * price
    if cost > s["usdt"]:
        return None
    sl = round(price - ATR_MULT * atr, 4)
    s["usdt"] -= cost
    s["coin"] += qty
    s["pos"] = {"entry": price, "qty": qty, "sl": sl, "atr": atr}
    s["n"] += 1
    return {"side": "BUY", "qty": qty, "price": price, "sl": sl, "atr": atr,
            "id": "PAPER-" + pair["name"] + "-" + str(s["n"]).zfill(4)}

def do_sell(pair, price, reason="SIGNAL"):
    s = states[pair["symbol"]]
    qty = s["coin"]
    if qty <= 0.000001:
        return None
    s["usdt"] += qty * price
    s["coin"] = 0.0
    pnl = pnl_pct = 0.0
    entry = None
    if s["pos"]:
        entry = s["pos"]["entry"]
        pnl = (price - entry) * qty
        pnl_pct = (price - entry) / entry * 100
        s["pnl"] += pnl
        if pnl >= 0:
            s["wins"] += 1
        else:
            s["loss"] += 1
        s["pos"] = None
    s["n"] += 1
    s["log"].append({"entry": entry, "exit": price, "pnl": pnl, "reason": reason,

```

```

        "t": datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M"))))
return {"side": "SELL", "qty": round(qty, 6), "price": price,
        "pnl": pnl, "pnl_pct": pnl_pct, "reason": reason,
        "id": "PAPER-" + pair["name"] + "-" + str(s["n"]).zfill(4)}

def check_sl(pair, price):
    s = states[pair["symbol"]]
    if s["pos"] and price <= s["pos"]["sl"]:
        logger.warning(pair["name"] + " SL at " + str(price))
        t = do_sell(pair, price, reason="STOP-LOSS")
        if t:
            tg(fmt(pair, t))
        return True
    return False

def fmt(pair, trade):
    s = states[pair["symbol"]]
    ts = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC")
    side = trade["side"]
    reason = trade.get("reason", "SIGNAL")
    if reason == "STOP-LOSS":
        em, hd = "&#x26A0;", "STOP-LOSS TRIGGERED"
    elif side == "BUY":
        em, hd = "&#x1F7E2;", "BUY Signal"
    else:
        em, hd = "&#x1F534;", "SELL Signal"
    lines = [
        em + " <b>" + hd + "</b>",
        "&#x1F4C5; " + ts,
        "&#x1F4E6; PAPER v2.0",
        "&#x1F4B1; " + pair["name"] + "/USDT",
        "&#x1F4B0; Price: $" + format(trade["price"], ".4f"),
        "&#x1F4CA; Qty: " + str(trade["qty"]) + " " + pair["name"],
    ]
    if side == "BUY":
        lines += [
            "&#x1F6E1; SL: $" + format(trade["sl"], ".4f") + " (ATR" + str(ATR_)
            "&#x26A0; Risk: " + str(RISK_PERCENT) + "%",
        ]
    if side == "SELL" or reason == "STOP-LOSS":
        pnl = trade.get("pnl", 0)
        pnl_pct = trade.get("pnl_pct", 0)
        icon = "&#x2705;" if pnl >= 0 else "&#x274C;"
        lines += [
            icon + " PnL: $" + format(pnl, "+.2f") + " (" + format(pnl_pct, "+.2f"

```

```

        "&#x1F4CB; Reason: " + reason,
    ]
    lines += [
        "&#x1F4BC; Balance: $" + format(s["usdt"], ",.2f") + " USDT",
        "&#x1F4C8; Total PnL: $" + format(s["pnl"], "+.2f"),
        "&#x1F3C6; W:" + str(s["wins"]) + " L:" + str(s["loss"]),
        "&#x1F3AB; " + trade["id"],
    ]
    return "\n".join(lines)

def daily_summary():
    lines = ["&#x1F4CA; <b>Daily Summary v2.0</b>", ""]
    tot_pnl = tw = tl = 0
    for pair in PAIRS:
        s = states[pair["symbol"]]
        tot_pnl += s["pnl"]
        tw += s["wins"]
        tl += s["loss"]
        wr = round(s["wins"] / max(s["wins"] + s["loss"], 1) * 100)
        pos = ""
        if s["pos"]:
            pos = " | Open @ $" + format(s["pos"]["entry"], ",.2f")
        lines.append("<b>" + pair["name"] + "</b>: $" + format(s["usdt"], ",.2f")
            + " | PnL: $" + format(s["pnl"], "+.2f") +
            " | W:" + str(s["wins"]) + " L:" + str(s["loss"]) +
            " (" + str(wr) + "%)" + pos)
    twr = round(tw / max(tw + tl, 1) * 100)
    lines += [
        "",
        "&#x1F4B0; <b>Total PnL: $" + format(tot_pnl, "+.2f") + "</b>",
        "&#x1F3C6; W:" + str(tw) + " L:" + str(tl) + " (" + str(twr) + "%)",
        "&#x2699; EMA50/200 + Williams%R + MACD + ATR",
    ]
    return "\n".join(lines)

def run_bot():
    logger.info("Bot v2.0 starting")
    msg = (
        "&#x1F916; <b>Advanced Bot v2.0 Started</b>\n"
        "Mode: PAPER TRADING\n"
        "Pairs: BTC | ETH | SOL\n"
        "Strategy: EMA50/200 + Williams%R + MACD + ATR\n"
        "Timeframe: 30min | Risk: 2%\n"
        "Balance: $3,333 per pair\n\n"
        "Improvements vs v1.0:\n"
    )

```

```

        "- EMA200 trend filter\n"
        "- Williams %R entry\n"
        "- MACD confirmation\n"
        "- Dynamic ATR stop-loss\n"
        "- 30min timeframe"
    )
    tg(msg)

n = 0
while True:
    n += 1
    logger.info("Check #" + str(n))
    for pair in PAIRS:
        try:
            df = get_klines(pair["yahoo"])
            if df is None:
                continue
            df = indicators(df)
            price = get_price(pair["yahoo"])
            if price is None:
                continue
            c = df.iloc[-1]
            up = c["ef"] > c["es"]
            logger.info(
                pair["name"] + " $" + format(price, ",.2f") +
                " EMA50=" + format(c["ef"], ".2f") +
                " EMA200=" + format(c["es"], ".2f") +
                " WR=" + format(c["wr"], ".1f") +
                " MACD=" + format(c["mh"], ".4f") +
                " ATR=" + format(c["atr"], ".2f") +
                " Trend=" + ("UP" if up else "DOWN")
            )
            s = states[pair["symbol"]]
            if s["pos"]:
                if check_sl(pair, price):
                    continue
            sig = signal(df)
            if sig == "BUY":
                t = do_buy(pair, price, c["atr"])
                if t:
                    tg(fmt(pair, t))
                    logger.info(pair["name"] + " BUY @ $" + str(price))
            elif sig == "SELL" and s["pos"]:
                t = do_sell(pair, price, reason="SIGNAL")
                if t:
                    tg(fmt(pair, t))
                    logger.info(pair["name"] + " SELL @ $" + str(price))

```

```
        except Exception as e:
            err = "&#x274C; <b>" + pair["name"] + " Error</b>\n" + str(e)
            logger.error(err)
            tg(err)
            time.sleep(2)

    if n % 48 == 0:
        tg(daily_summary())

    logger.info("Sleeping 30 min")
    time.sleep(CHECK_INTERVAL)

if __name__ == "__main__":
    run_bot()
```